МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Казанский национальный исследовательский технический университет им. А.Н. Туполева – КАИ»
Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности

# Лабораторная работа № 1

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «Основы OpenMP: параллельные области, распределение работ, синхронизация потоков»

Работу выполнил
Студент гр. 4404
Завалин М.Г.

Казань, 2026

**Цель работы:**

Изучение основ параллельного программирования для систем с общей памятью: параллельные области и идентификация потоков, распределение итераций цикла между потоками (типы расписания schedule), примитивы синхронизации потоков. В ходе работы выполнены задачи 1–3 методических указаний «Параллельное программирование на OpenMP». Программы написаны на языке C с использованием директив компилятора и библиотечных функций OpenMP (pragma omp, omp_get_thread_num, omp_set_num_threads и др.).

## 1. Задание

Задача 1. Написать программу, где каждый поток печатает свой идентификатор, количество потоков всего и строчку «Hello World». Запустить программу с 8 потоками. Всегда ли вывод идентичен? Почему?

Задача 2. Написать программу, в которой объявлен массив из 16000 элементов, инициализированный так, что значение элемента равно его порядковому номеру. Создать результирующий массив, в котором (за исключением крайних элементов) будут средние значения исходного массива: b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0. Запустить программу с 8 потоками при различных типах распределения работ.

Задача 3. Модифицировать задачу 1 так, чтобы потоки печатали свои идентификаторы в обратном порядке. Существует как минимум 5 способов решения; необходимо реализовать и продемонстрировать как можно больше из них.

Общие требования: число потоков и размер задачи передаются программе через командную строку; запрещается использовать библиотечные функции, целиком

решающие задачу; каждый запуск, о котором говорится в задаче (разное число потоков, разные типы распределения, разные версии программы), подтверждается скриншотом с видимой командой запуска и полным выводом.

## 2. Ход работы

### 2.1. Задача 1. Параллельная область и идентификация потоков

Программа запрашивает у пользователя число потоков N, затем с помощью директивы #pragma omp parallel num_threads(N) создаёт параллельную область из N потоков. Каждый поток получает свой идентификатор вызовом omp_get_thread_num() и общее число потоков вызовом omp_get_num_threads(), после чего печатает строку вида «Thread X of Y: Hello World».

**Код программы:**

```
#include <stdio.h>
#include <omp.h>

int main() {
    printf("Vvedite chislo potokov \n");
    int N = 0;
    scanf_s("%d", &N);

#pragma omp parallel num_threads(N)
    {
        int tid = omp_get_thread_num();
        int nthreads = omp_get_num_threads();
        printf("Thread %d of %d: Hello World\n", tid, nthreads);
    }

    return 0;
}
```

Ответ на вопрос задачи: вывод не всегда идентичен. Потоки выполняются параллельно, и порядок, в котором они достигают оператора printf, определяется планировщиком операционной системы, поэтому от запуска к запуску порядок строк может меняться. Множество строк (по одной от каждого потока) одинаково во всех запусках, но их последовательность программой не задаётся, так как синхронизация, фиксирующая порядок вывода, отсутствует.

Для подтверждения выполнено три запуска программы с 8 потоками. На рисунке 1 показан первый запуск программы с 8 потоками. Видна команда запуска и полный вывод: каждый из 8 потоков напечатал свой идентификатор, общее число потоков и строку «Hello World».

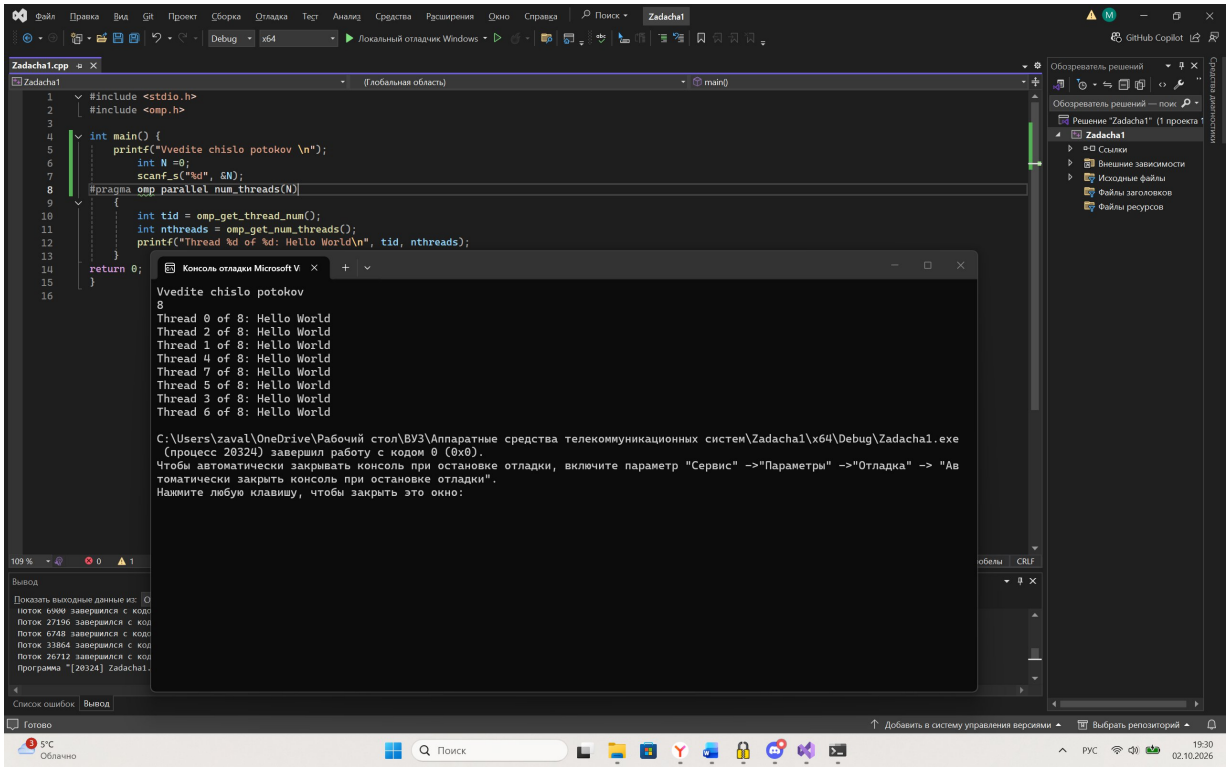



*Рисунок 1 — Задача 1: запуск с 8 потоками, запуск № 1*

На рисунке 2 показан второй запуск той же команды. Порядок строк может отличаться от рисунка 1, так как порядок выполнения потоков не определён. Если порядок совпал, это не гарантирует детерминированность вывода — программа порядок не фиксирует.

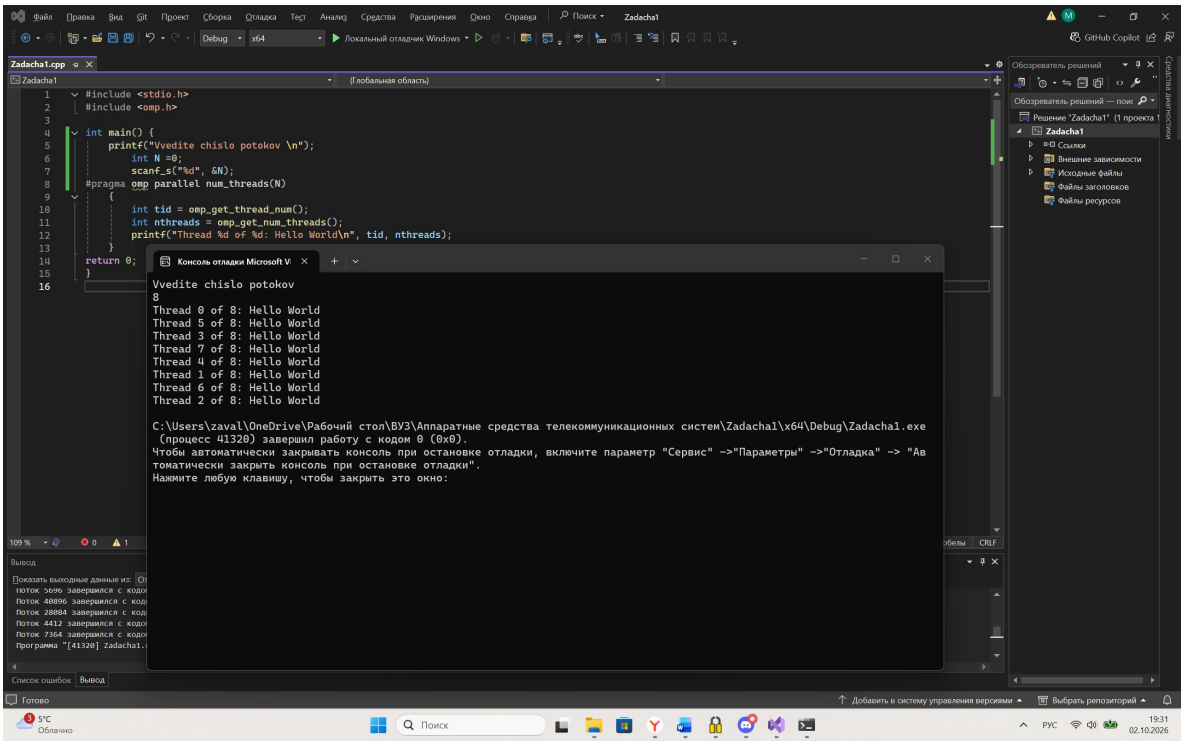



*Рисунок 2 — Задача 1: запуск с 8 потоками, запуск № 2*

На рисунке 3 показан третий запуск. Сравнение рисунков 1–3 показывает, что набор строк вывода во всех запусках одинаков, а их порядок может изменяться, что подтверждает ответ на вопрос задачи.

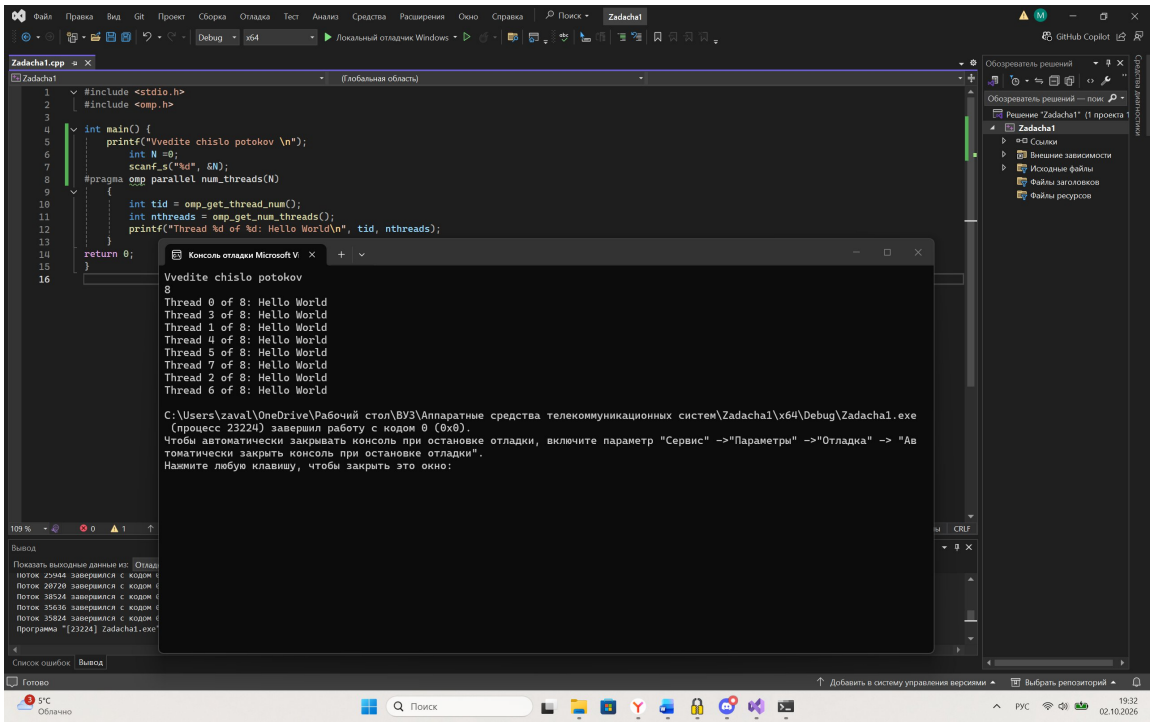



*Рисунок 3 — Задача 1: запуск с 8 потоками, запуск № 3*

### 2.2. Задача 2. Обработка массива при различных типах распределения работ.

Написать программу, в которой объявлен массив из 16000 элементов и инициализирован так, что значение элемента массива равно его порядковому номеру. Затем создайте результирующий массив, в котором (за исключением крайних элементов) будут средние значения исходного массива: b[i] = (a[i-1] + a[i] + a[i+1])/3.0. Запустите программу с 8-ю процессами при различных типах распределения работ.

**Код программы:**

```
#include <stdio.h>
#include <omp.h>
#define N 16000 // во всём коде N заменяется 16000
int main() {
   int a[N], b[N]; // 2 массива
   int i; // счётчик цикла
   printf("Vvedite chislo potokov \n");
   int npotok = 0;
   scanf_s("%d", &npotok);
   // Инициализация: a[i] = i
   for (i = 0; i < N; i++)
      a[i] = i;
   // Крайние элементы обрабатываем отдельно
   b[0] = a[0];
   b[N - 1] = a[N - 1];
   omp_set_num_threads(npotok);
   // --- static ---
#pragma omp parallel for shared(a, b) private(i) schedule(static)
   for (i = 1; i < N - 1; i++)
      b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
   printf("static:  b[1]=%d  b[7999]=%d  b[15998]=%d\n",
      b[1], b[7999], b[15998]);
   // --- dynamic ---
#pragma omp parallel for shared(a, b) private(i) schedule(dynamic, 100)
```

```
    for (i = 1; i < N - 1; i++)
        b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
    printf("dynamic: b[1]=%d  b[7999]=%d  b[15998]=%d\n",
        b[1], b[7999], b[15998]);
    // --- guided ---
#pragma omp parallel for shared(a, b) private(i) schedule(guided, 100)
    for (i = 1; i < N - 1; i++)
        b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
    printf("guided:  b[1]=%d  b[7999]=%d  b[15998]=%d\n",
        b[1], b[7999], b[15998]);
    // --- runtime ---
#pragma omp parallel for shared(a, b) private(i) schedule(runtime)
    for (i = 1; i < N - 1; i++)
        b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
    printf("runtime: b[1]=%d  b[7999]=%d  b[15998]=%d\n",
        b[1], b[7999], b[15998]);
    return 0;
}
```

Программа обрабатывает 15998 внутренних итераций (i = 1 … N-2) и использует #pragma omp parallel for с клаузой schedule(...) для распределения этих итераций между потоками. Всего в коде применены четыре типа расписания:

schedule(static)

Итерации заранее, на этапе компиляции, делятся между потоками непрерывными равными (или почти равными) блоками. При 8 потоках и 15998 итерациях каждый поток получает примерно 2000 итераций подряд — например, поток 0 обрабатывает индексы 1–2000, поток 1 — 2001–4000 и т. д. Распределение фиксируется до начала цикла, накладные расходы на планирование минимальны. Недостаток: если отдельные итерации выполняются за разное время, часть потоков завершит работу раньше и будет простаивать. В данной задаче все итерации одинаковы по трудоёмкости, поэтому static работает эффективно.

schedule(dynamic, 100)

Итерации разбиваются на блоки фиксированного размера (в данном случае по

100). Поток берёт блок из общей очереди, обрабатывает его, затем запрашивает следующий. Если один поток работает медленнее, другие заберут оставшиеся блоки — балансировка загрузки происходит автоматически. Поскольку очередь общая, доступ к ней защищён механизмом синхронизации (аналог критической секции внутри реализации OpenMP), что создаёт дополнительные накладные расходы по сравнению со static. Чем меньше размер блока, тем лучше балансировка, но больше overhead на выдачу блоков.

schedule(guided, 100)

Как и dynamic, потоки берут блоки из общей очереди по мере освобождения. Отличие в размере блока: первые блоки крупные (примерно «оставшаяся работа / число потоков»), а каждый следующий блок уменьшается по мере обработки, но не становится меньше указанного минимума (100). Благодаря этому в начале работы накладные расходы на синхронизацию минимальны (крупные блоки), а в конце — балансировка хорошая (мелкие блоки дораздают остатки). guided — компромисс между static (нет накладных расходов на очередь, но нет балансировки) и dynamic (хорошая балансировка, но много синхронизации).

schedule(runtime)

Тип расписания и размер блока не задаются в коде, а определяются во время запуска через переменную окружения OMP_SCHEDULE. Например, если перед запуском выполнить set OMP_SCHEDULE=dynamic,100 (Windows) или export OMP_SCHEDULE=guided,50 (Linux), то цикл будет выполнен с соответствующим расписанием. Если переменная не задана, по стандарту OpenMP используется static. Это позволяет менять стратегию распределения без перекомпиляции программы, что удобно для экспериментов.

Работа программы представлена на рисунке 4.

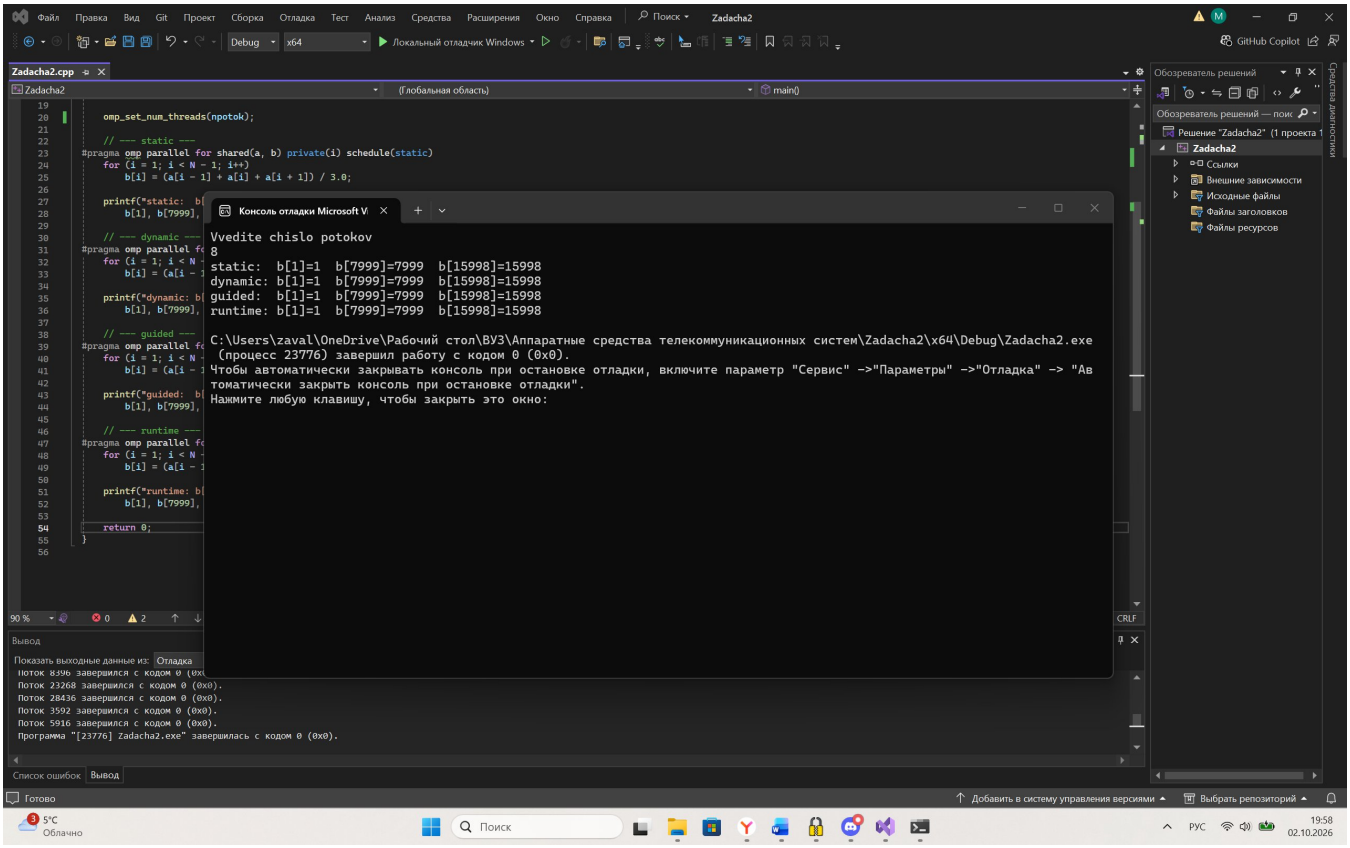



*Рисунок 4 — Задача 2: вывод программы*

Во всех четырёх случаях результат одинаков: b[1] = 1, b[7999] = 7999, b[15998] = 15998. Итерации цикла независимы — каждая итерация i читает элементы a[i-1], a[i], a[i+1] из неизменяемого массива a и пишет только в свой элемент b[i]. Гонки данных нет, а значит, итоговый массив b определяется только формулой и не зависит от того, в каком порядке потоки обрабатывают итерации. Тип расписания влияет только на то, какие именно итерации достаются каждому потоку и сколько времени занимает выполнение, но не на результат.

### 2.3. Задача 3. Вывод идентификаторов потоков в обратном порядке

Для фиксации порядка вывода необходимы средства синхронизации. В задаче требуется модифицировать программу из задачи 1 так, чтобы потоки печатали свои идентификаторы в обратном порядке: от N-1 до 0.

**1 способ** - активное ожидание на атомарном счётчике (atomic). Программа считывает число потоков и инициализирует разделяемую переменную next = npotok - 1. Все потоки запускаются одновременно в параллельной области. Каждый поток в цикле проверяет, не достигла ли переменная next его идентификатора. Когда next == tid, поток печатает свою строку и атомарно уменьшает next на единицу, передавая очередь следующему потоку. Атомарная операция #pragma omp atomic гарантирует, что декремент выполнится без гонки данных. Работа программы представлена на рисунке 5.

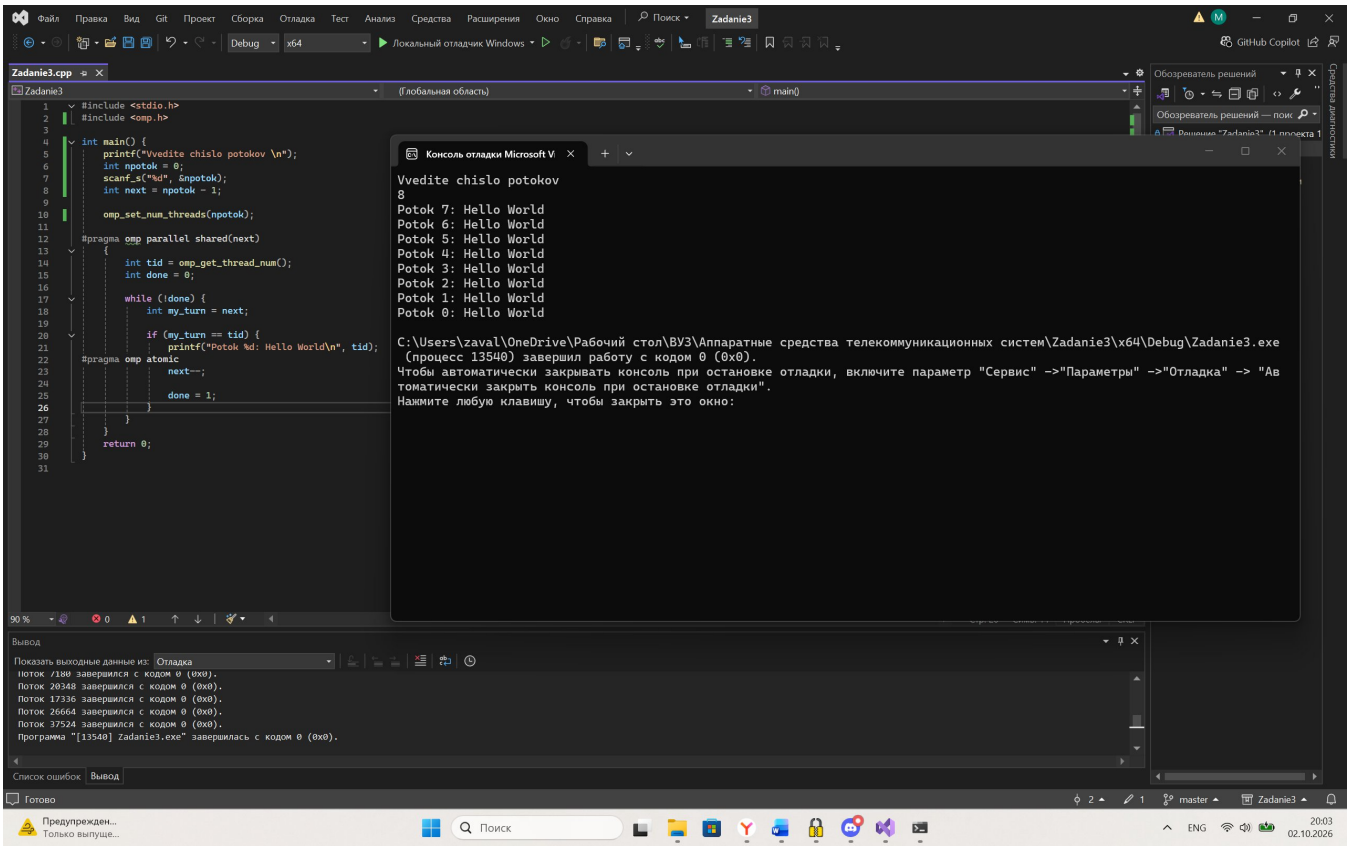



*Рисунок 5 — Задача 3: вывод программы, активное ожидание (spinlock) на атомарном счётчике*

**2 способ** - критическая секция (critical). Логика та же, что и в способе 1: разделяемая переменная next инициализируется значением N-1, и каждый поток

ждёт, пока next не совпадёт с его tid. Отличие — в механизме синхронизации: вместо атомарной операции используется критическая секция #pragma omp critical. Поток входит в критическую секцию, проверяет условие next == tid и, если оно истинно, печатает строку, уменьшает next и выходит из цикла. Критическая секция гарантирует, что только один поток одновременно выполняет блок кода внутри неё, поэтому доступ к next защищён от гонки данных.

По сравнению с атомарным счётчиком критическая секция имеет более высокие накладные расходы: каждый вход и выход сопровождается захватом и освобождением блокировки (аналог мьютекса), тогда как #pragma omp atomic обеспечивает только неделимость одной операции. Однако в данном случае это не критично, так как основное время поток проводит в цикле ожидания вне критической секции — проверка next == tid выполняется до входа в critical только в способе 1, а в способе 2 проверка находится внутри. Работа программы представлена на рисунке 6.

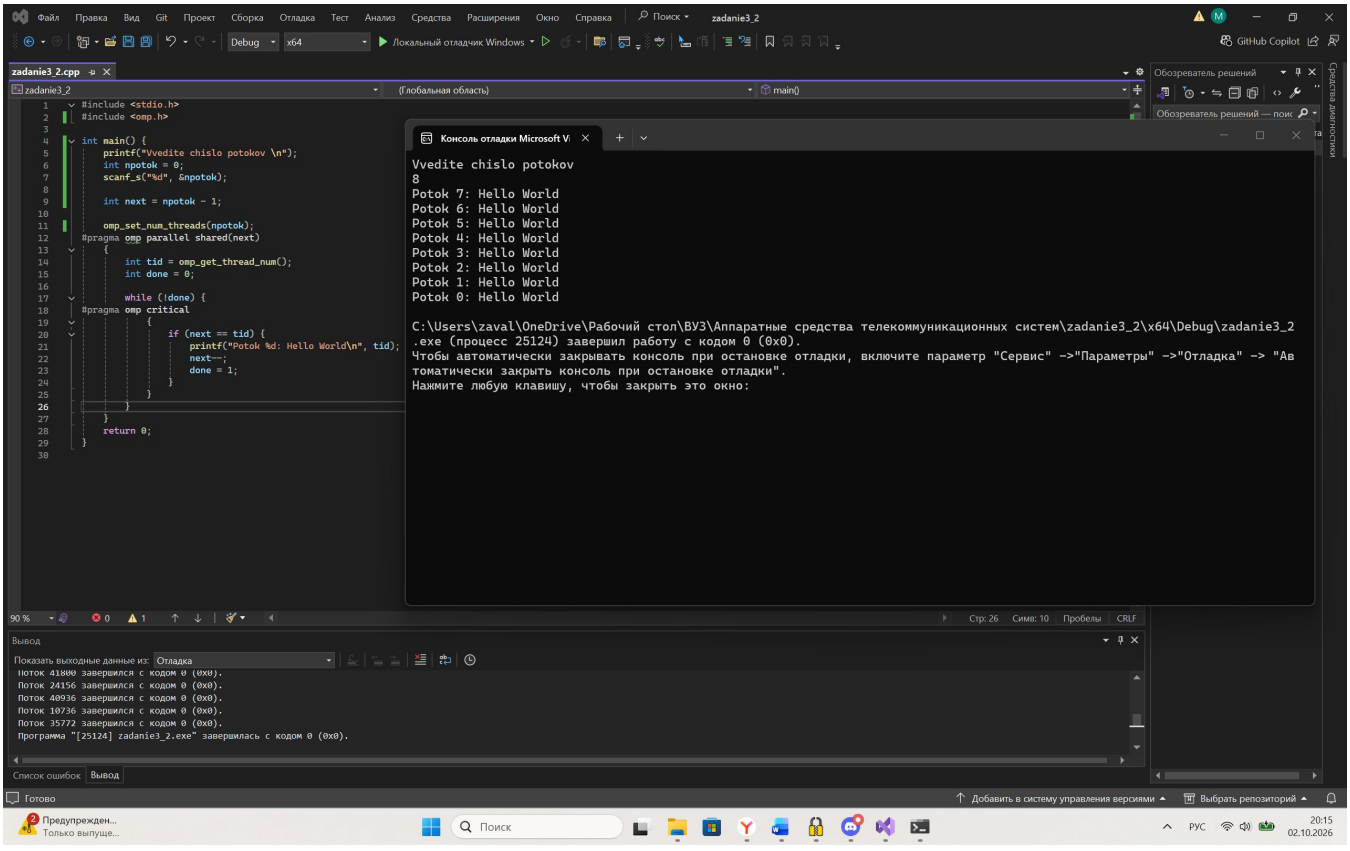



*Рисунок 6 — Задача 3: вывод программы, критическая секция*

**3 способ** - Цепочка барьеров (barrier). В этом способе не используются ни атомарные операции, ни критические секции — только директива #pragma omp barrier. Все потоки входят в параллельную область и запускают общий цикл for (step = nthreads - 1; step >= 0; step--). На каждой итерации цикла:

1. Поток, у которого tid == step, печатает свою строку.
2. Все потоки доходят до барьера #pragma omp barrier и ждут, пока все остальные тоже достигнут этой точки.
3. После барьера цикл переходит к следующему шагу (step--).

Таким образом, на первом шаге (step = 7) печатает поток 7, затем все потоки синхронизируются на барьере, после чего на втором шаге (step = 6) печатает поток 6, и так далее до потока 0. Барьер гарантирует, что ни один поток не начнёт следующую итерацию, пока все не завершили текущую, — это и обеспечивает строгий порядок печати. Работа программы представлена на рисунке 7.

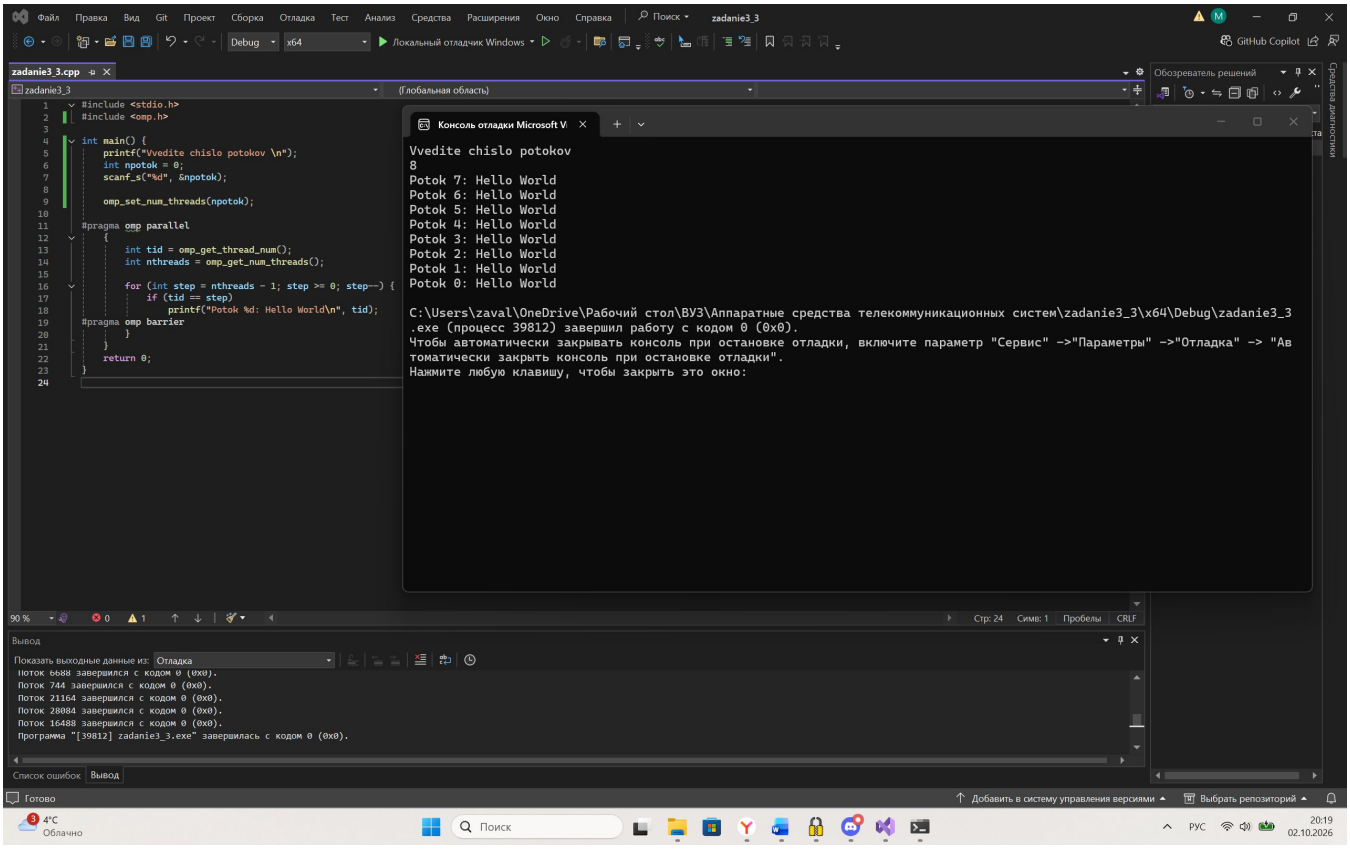



*Рисунок 7 — Задача 3: вывод программы, цепочка барьеров*

**4 способ** – замки (omp_lock_t). Логика та же, что и в способах 1–2: разделяемая переменная next инициализируется значением N-1, и каждый поток ждёт, пока next не совпадёт с его tid. Отличие — в механизме синхронизации: вместо атомарной операции или критической секции используется явный замок типа omp_lock_t. Замок создаётся функцией omp_init_lock(), захватывается omp_set_lock() и освобождается omp_unset_lock(). Поток захватывает замок, проверяет условие next == tid и, если оно истинно, печатает строку, уменьшает next и освобождает замок. Если условие ложно — замок просто освобождается, и поток продолжает цикл ожидания. После завершения параллельной области замок уничтожается функцией omp_destroy_lock().

По сравнению с #pragma omp critical (способ 2) явный замок omp_lock_t даёт больше контроля: замок можно инициализировать и уничтожить вручную, передавать в функции, использовать несколько независимых замков. По производительности для одной точки синхронизации различия между critical и omp_lock_t минимальны — обе механизма реализованы через мьютексы ОС. Работа программы представлена на рисунке 8.

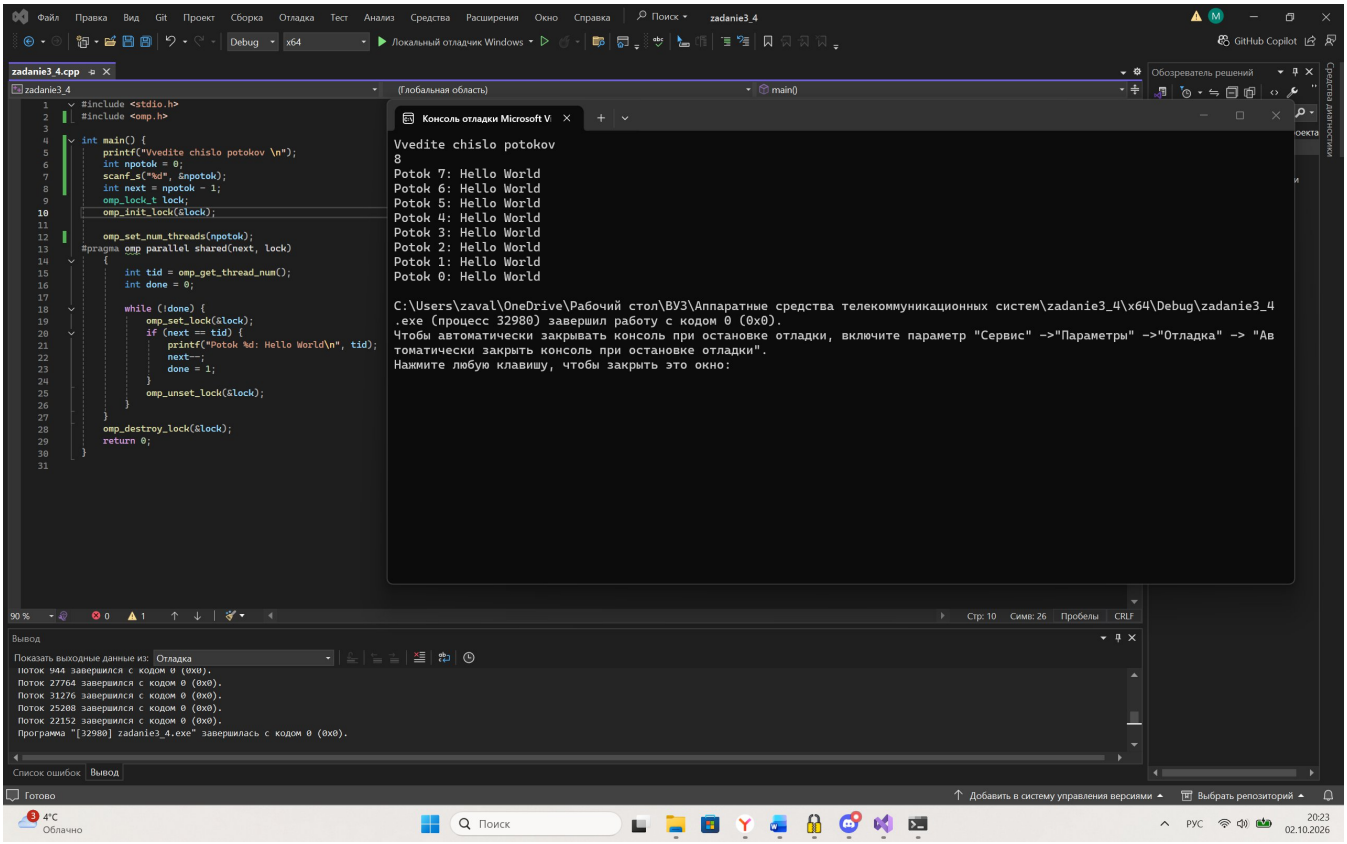



*Рисунок 8 — Задача 3: вывод программы, замки*

5 способ – single. В этом способе используется принципиально иной подход: вместо того чтобы каждый поток печатал свою строку в нужный момент, потоки сначала собирают свои идентификаторы в разделяемый массив ids[], синхронизируются барьером, а затем один поток (#pragma omp single) печатает все идентификаторы в обратном порядке.

Каждый поток записывает свой tid в ids[tid], после чего все потоки достигают барьера #pragma omp barrier — это гарантирует, что массив полностью заполнен. Затем директива #pragma omp single выбирает один из потоков (произвольно), который выполняет цикл от npotok - 1 до 0 и печатает все строки. Остальные потоки пропускают блок single и ждут на неявном барьере в конце single. Работа программы представлена на рисунке 9.

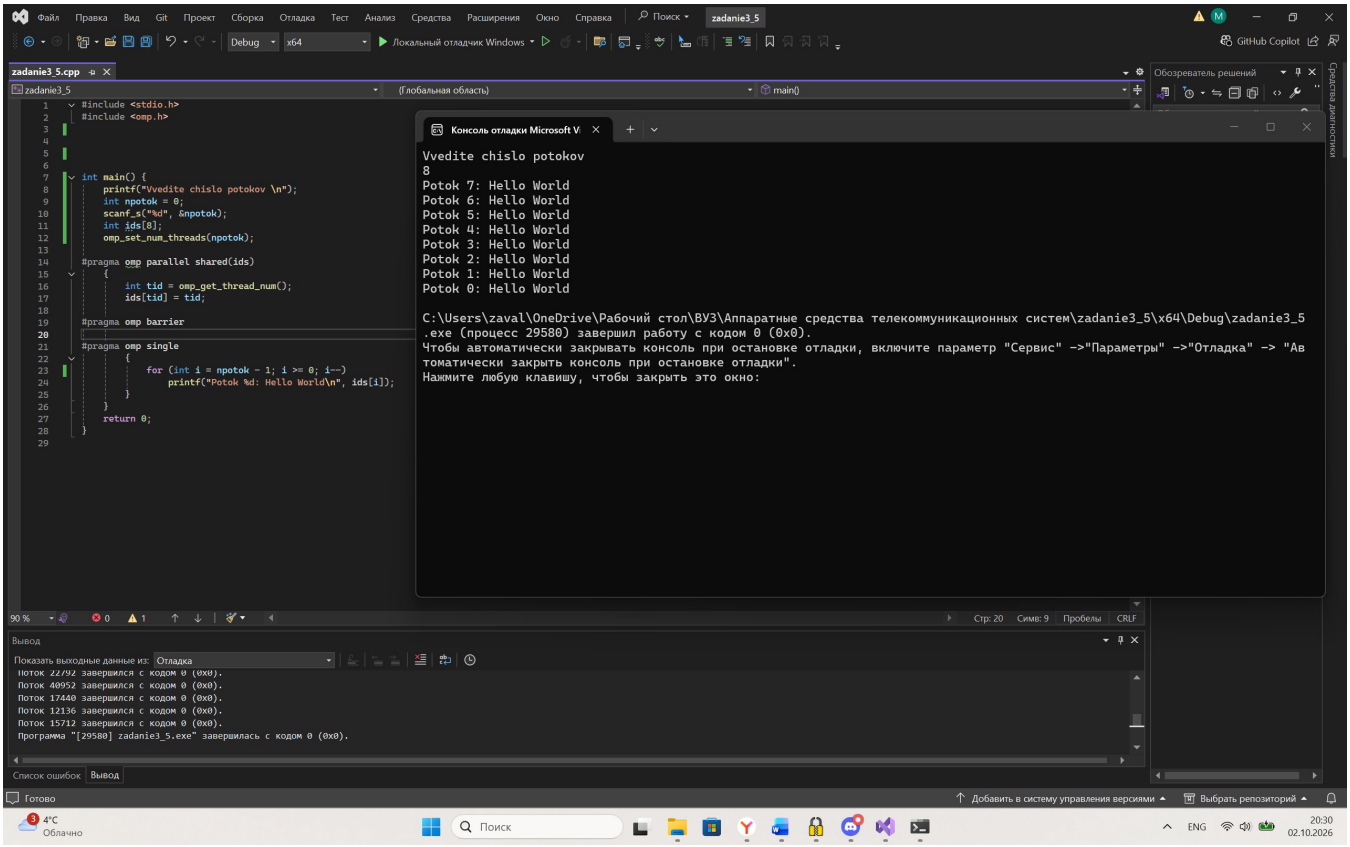



*Рисунок 9 — Задача 3: вывод программы, single*